

ARTIFICIAL LIFE

Simulation Using Neuroevolution and Genetic Algorithms

System Design Document

Deep Learning - Evolutionary Computation - Artificial Life

DOCUMENT PURPOSE

This document defines the functional, technical, algorithmic, architectural, experimental, and deployment design of the proposed artificial-life simulation. It is written as the engineering baseline for implementation and academic evaluation.

Item	Value
Document version	1.0
Project type	Deep Learning / AI / Simulation
Primary language	Python
Simulation engine	PyGame
Neural framework	PyTorch
Evolution	Custom Genetic Algorithm; NEAT-style extension
Target platform	Desktop; optional web dashboard
Status	Design baseline

Table of Contents

1. Executive Summary
 2. Problem Definition
 3. Objectives and Success Criteria
 4. Scope and Constraints
 5. System Concept
 6. Functional Requirements
 7. Non-Functional Requirements
 8. Architecture
 9. Environment Design
 10. Agent and Sensor Design
 11. Neural Network Design
 12. Genetic Algorithm Design
 13. Fitness and Reward Design
 14. Reproduction and Species
 15. Simulation Engine and Runtime Loop
 16. UI/UX and Analytics Dashboard
 17. Data and Experiment Management
 18. Experimental Methodology
 19. Evaluation Plan
 20. Development Plan
 21. Testing Strategy
 22. Risk Analysis
 23. Security and Stability
 24. Deployment and Packaging
 25. Future Extensions
 26. Expected Deliverables
 27. Conclusion
- Appendix A - Recommended Initial Parameters
- Appendix B - Reference Pseudocode
- Appendix C - Proposed Project Title Variants

1. Executive Summary

The proposed system is a 2D artificial-life simulation in which autonomous organisms learn and evolve behaviors through neural networks and a genetic algorithm. The simulation replaces hand-authored behavior rules with adaptive controllers whose parameters are encoded in genomes and improved across generations according to a fitness function.

The initial implementation targets a manageable single-species ecosystem containing organisms, food, obstacles, energy, and limited sensory perception. The advanced implementation introduces predators, reproduction, species, dynamic environments, and NEAT-style topology evolution. A statistics dashboard records population behavior and evolutionary performance so that the project can be evaluated scientifically rather than only demonstrated visually.

CORE RESEARCH QUESTION

Can useful adaptive behavior emerge in autonomous agents when neural-network controllers are optimized by evolutionary selection rather than directly programmed?

2. Problem Definition

Traditional game and simulation agents are often controlled by explicit if/then rules. Such rules are effective for deterministic behavior, but they do not provide a mechanism for discovering new strategies. The project addresses this limitation by creating agents whose decision policy is learned indirectly: the neural network defines behavior, while evolution searches for network parameters that improve survival and task performance.

Rule-based approach

IF food is near -> move toward food

IF predator is near -> move away

Evolutionary approach

Sensors -> Neural Network -> Actions -> Fitness -> Selection -> Mutation -> New generation

3. Objectives and Success Criteria

Objective	Success criterion
Environment	Stable real-time 2D world with deterministic and randomized configurations.
Autonomous agent	Agent acts from sensor observations without hard-coded task policy.
Neural controller	Controller maps normalized observations to movement/action outputs.
Evolution	Fitness-driven selection, crossover, mutation, and generation replacement work reproducibly.
Learning evidence	Median fitness improves across generations under controlled seeds.
Generalization	Evolved agents can be evaluated on unseen environment layouts.
Analytics	Dashboard reports fitness, survival, resource use, diversity, and runtime statistics.
Engineering quality	Automated tests cover core modules and experiments are

	reproducible from configuration files.
--	--

4. Scope and Constraints

4.1 In Scope

- 2D continuous simulation space.
- Autonomous organisms with limited local perception.
- Food/resource seeking and obstacle avoidance.
- Energy, health, age, movement cost, and death.
- Genetic representation of neural-network parameters.
- Selection, elitism, crossover, mutation, and generation management.
- Experiment logging and statistical plots.
- Optional predator/prey extension and species grouping.

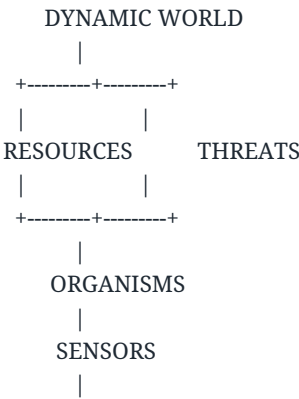
4.2 Out of Scope for the First Release

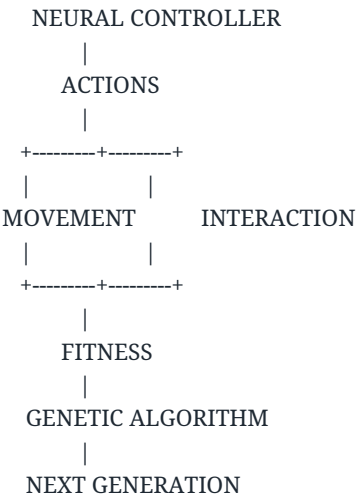
- Photorealistic 3D environments.
- Real-world robot hardware integration.
- Large-scale distributed evolution across multiple machines.
- Biologically accurate genetics or ecology.
- Medical or safety-critical decision making.

4.3 Constraints

Constraint	Design response
Compute budget	Begin with 100-250 agents and lightweight dense neural networks.
Real-time rendering	Separate simulation update rate from rendering frame rate.
Reproducibility	Use explicit random seeds and versioned experiment configurations.
Complexity control	Implement features in staged releases from MVP to advanced NEAT ecosystem.

5. System Concept





The system is a closed evolutionary loop. Agents perceive local state, the neural controller converts observations into actions, the simulation evaluates the consequences, and the genetic algorithm uses fitness to create the next population.

6. Functional Requirements

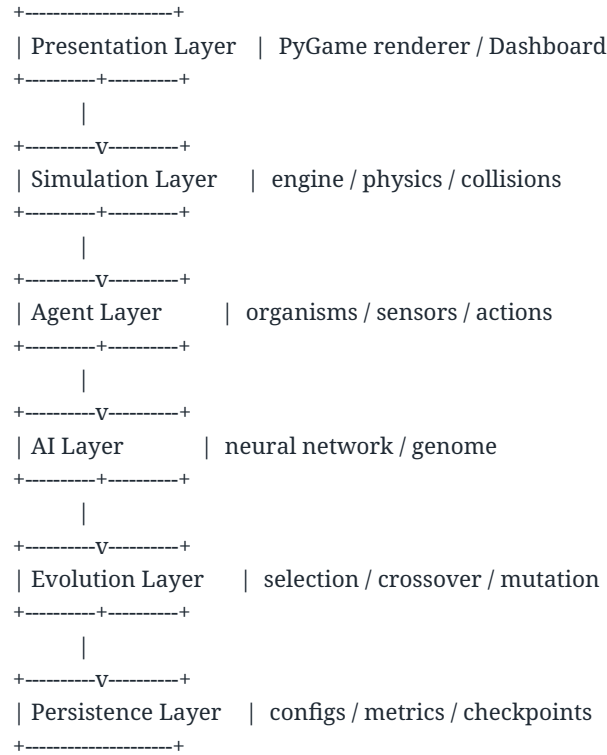
ID	Requirement
FR-01	Initialize environment with configurable dimensions, resource density, obstacles, and population size.
FR-02	Spawn organisms with valid genomes and deterministic identifiers.
FR-03	Calculate sensor observations from nearby entities and obstacles.
FR-04	Run the neural controller and generate continuous action outputs.
FR-05	Apply physics, movement costs, collisions, eating, damage, and death.
FR-06	Accumulate per-agent fitness during each generation.
FR-07	End a generation when a configured time limit or terminal condition is reached.
FR-08	Select parents using configurable selection strategy.
FR-09	Generate offspring through crossover and mutation.
FR-10	Log generation-level and agent-level metrics.
FR-11	Pause, resume, accelerate, reset, and save the simulation.
FR-12	Run predefined experiments from configuration files without UI interaction.

7. Non-Functional Requirements

Category	Requirement
Performance	Target stable simulation at configurable real-time FPS for the baseline population.
Reproducibility	Same seed + same configuration should produce equivalent experiment results.
Maintainability	Separate environment, agent, neural, evolution, visualization, and analytics modules.
Extensibility	New sensors, actions, organisms, or evolutionary

	strategies should be addable without rewriting the engine.
Observability	Runtime and evolution statistics must be accessible during and after experiments.
Robustness	Invalid values, NaN weights, malformed genomes, and empty populations must be handled safely.
Usability	Controls and statistics should remain understandable during live demonstrations.

8. Architecture



8.1 Module Responsibilities

Module	Responsibilities
simulation.engine	Main loop, time steps, generation lifecycle, orchestration.
simulation.environment	World state, resources, obstacles, spawn/reset logic.
simulation.physics	Position, velocity, steering, energy consumption, collision responses.
agents.organism	Agent state, genome binding, actions, lifecycle.
agents.sensors	Ray casting, target detection, normalized observations.
neural.network	Forward pass and configurable architecture.
neural.genome	Serialization and genome vector representation.
evolution.genetic_algorithm	Population evolution and generation creation.
evolution.selection	Tournament, roulette, rank, and elitism strategies.
evolution.crossover	One-point, uniform, or arithmetic crossover.
evolution.mutation	Gaussian mutation, clipping, adaptive mutation extension.
visualization.renderer	World rendering and entity visualization.
visualization.dashboard	Metrics, graphs, controls, best-agent inspector.
analytics	Experiment logging, aggregation, plots, and summary reports.

9. Environment Design

9.1 World Model

World(width, height)

- resources: list[Food]
- obstacles: list[Obstacle]
- agents: list[Organism]
- time: float
- generation: int
- random_seed: int

9.2 Food

Property	Description
position	2D coordinates
energy_value	Energy restored when consumed
respawn	Optional probability or timed respawn
radius	Collision/eating radius

9.3 Obstacles

Obstacles are static geometric objects in the MVP. The renderer should display them distinctly, while the sensor subsystem treats them as detectable hazards. The advanced version may introduce moving obstacles or changing terrain.

9.4 World Boundaries

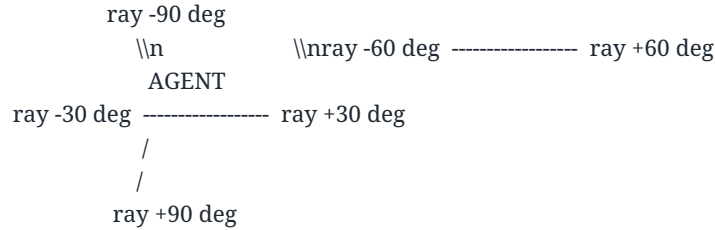
Agents are not allowed to leave the world. Boundary contact can either reflect velocity, clamp position, or reduce fitness. Clamping plus a small collision penalty is the recommended baseline because it avoids unstable reflections.

10. Agent and Sensor Design

10.1 Agent State

Field	Type	Purpose
id	int	Unique identifier
position	Vector2	Current location
velocity	Vector2	Movement state
heading	float	Orientation
energy	float	Survival resource
health	float	Damage state
age	int/float	Lifetime
fitness	float	Evolution score
genome	array	Neural parameters
alive	bool	Lifecycle state

10.2 Ray-Based Vision



Seven rays are recommended in the baseline configuration: -90, -60, -30, 0, +30, +60, +90 degrees. Each ray returns normalized features describing the nearest relevant object within a maximum range.

10.3 Sensor Encoding

Observation	Example encoding
Food distance	[0,1], where 0 is near and 1 is beyond max range
Food bearing	[-1,1], normalized left/right direction
Threat distance	[0,1]
Threat bearing	[-1,1]
Obstacle distance	[0,1]
Energy	[0,1]
Speed	[0,1]

The exact observation vector must remain fixed within an experiment so that genome dimensions stay compatible. Any change to input ordering is a new model configuration and experiment version.

10.4 Action Space

```

steering = tanh(output[0]) # -1 left to +1 right
accel    = sigmoid(output[1]) # 0 to 1
eat_signal = sigmoid(output[2]) # threshold for eating
  
```

11. Neural Network Design

The neural controller is a feed-forward multilayer perceptron for the baseline system. It is intentionally small so that hundreds of agents can be simulated concurrently. PyTorch is recommended for implementation because it provides clear tensor operations, serialization, and later extension to more advanced architectures.

```

Input: 12-24 normalized observations
|
Dense 32
| ReLU
Dense 16
| ReLU
Output layer 3
|
steering / accel / eat
  
```

Parameter	Baseline
Input size	12-24 depending on sensor configuration
Hidden 1	32 neurons

Hidden 2	16 neurons
Activation	ReLU
Output	3
Steering	tanh
Continuous actions	sigmoid/tanh as appropriate

11.1 Genome Encoding

All weights and biases are flattened into a one-dimensional vector. The neural-network class provides deterministic packing/unpacking so that the same genome always reconstructs the same controller.

Genome = [W1..., b1..., W2..., b2..., W3..., b3...]

Parent A + Parent B -> Child genome -> reshape -> neural network

11.2 Why a Fixed Network First?

A fixed topology isolates the evolutionary problem to parameter optimization. Once the baseline is validated, topology evolution can be added as a distinct experimental condition rather than mixing multiple sources of complexity from the beginning.

12. Genetic Algorithm Design

```

for generation in range(max_generations):
    simulate_population()
    evaluate_fitness()
    elites = select_elites(population)
    parents = select_parents(population)
    children = crossover_and_mutate(parents)
    population = elites + children
    log_metrics()

```

12.1 Selection

Tournament selection is recommended because it is simple, configurable, and less sensitive to extreme fitness scales than raw roulette-wheel selection. A small elite set is preserved unchanged to prevent loss of the best solution found so far.

12.2 Crossover

Method	Description
One-point	Take a prefix from parent A and suffix from parent B.
Uniform	Each gene is chosen independently from either parent.
Arithmetic	Weighted average for numeric genes.

Uniform crossover is the preferred baseline for dense neural-network genomes. One-point crossover can be tested as an experimental alternative.

12.3 Mutation

```

for gene in genome:
    if random() < mutation_rate:

```

```
gene += Normal(0, mutation_strength)
gene = clip(gene, min_weight, max_weight)
```

Mutation prevents premature convergence and introduces new candidate behaviors. Mutation rate and strength are configuration parameters and are therefore key experimental variables.

13. Fitness and Reward Design

Fitness defines what the evolutionary system considers successful. It must reward useful behavior without creating exploitable shortcuts. The recommended baseline combines survival, resource acquisition, exploration, and penalties.

```
fitness = (survival_time * 0.50)
         + (food_consumed * 20.0)
         + (distance_explored * 0.10)
         + (successful_reproduction * 50.0)
         - (collisions * 10.0)
```

DESIGN RULE
 Treat the coefficients as experimental parameters, not universal constants. A core research objective is to test whether different reward balances change the evolutionary outcome.

13.1 Fitness Failure Modes

Failure mode	Example	Mitigation
Reward hacking	Agent spins to maximize movement score.	Cap exploration contribution and require meaningful displacement.
Stalling	Agent stays still to avoid collision.	Energy drain for time and reward for resource collection.
Safe corner	Agent camps beside food indefinitely.	Randomize food positions and introduce periodic resource relocation.
Over-aggression	Predators waste energy chasing unreachable targets.	Include energy and capture efficiency in fitness.

14. Reproduction and Species

14.1 Reproduction

The MVP can use generation-level replacement without physical reproduction in the environment. The advanced version adds explicit reproduction where healthy, mature agents produce offspring. This separates the concept of genetic inheritance from the population-management algorithm and allows both to be studied.

```
eligible = age >= MIN_AGE and energy >= MIN_ENERGY and fitness >= MIN_FITNESS
child = crossover(parent_a.genome, parent_b.genome)
child = mutate(child)
```

14.2 Species

Species grouping is a future extension used to preserve diversity. Genomes are clustered using a compatibility distance based on parameter or topology differences. Separate species quotas can prevent one early strategy from eliminating all alternative strategies.

15. Simulation Engine and Runtime Loop

INITIALIZATION

- load configuration
- set random seed
- create world
- create population

EACH FRAME

- read controls
- update sensors
- run neural networks
- apply actions
- update physics
- resolve interactions
- update energy / health / age
- accumulate fitness
- render

GENERATION END

- finalize fitness
- log statistics
- select parents
- crossover
- mutate
- create next generation
- reset world or randomize according to experiment

15.1 Time Management

Mechanism	Purpose
Simulation timestep	Stable physics and deterministic progression
Render FPS	Controls visualization frequency
Fast-forward factor	Allows accelerated evolution during experiments
Generation duration	Bounds computational cost per generation

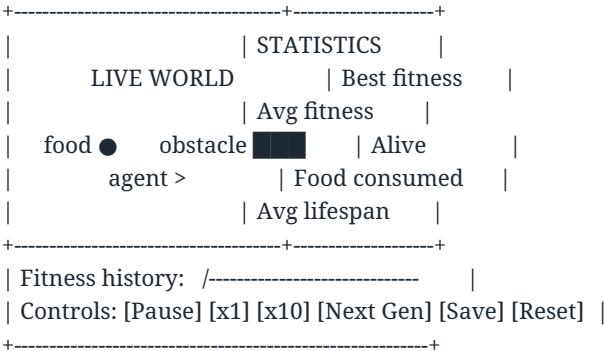
15.2 Determinism

Every experiment records the random seed, configuration hash, software version, and start timestamp. A deterministic simulation step is preferred wherever possible so that runs can be reproduced and compared.

16. UI/UX and Analytics Dashboard

+-----+

| GENERATION 384 | POPULATION 250 | SPECIES 6 |



16.1 Best-Agent Inspector

- Current fitness, age, energy, health, and movement speed.
- Sensor rays and current observations.
- Neural-network graph with active outputs.
- Genome identifier and parent lineage.
- Behavior timeline for the current generation.

17. Data and Experiment Management

Although the simulation generates its own experience data, persistent experiment logs are essential for scientific evaluation. Each run should be self-describing and independently reproducible.

```
experiments/  
EXP-001/  
  config.json  
  metadata.json  
  generation_metrics.csv  
  agent_metrics.csv  
  best_genome.json  
  checkpoints/  
EXP-002/  
...
```

17.1 Generation Metrics

Field	Description
generation	Generation index
best_fitness	Highest fitness
mean_fitness	Population mean
median_fitness	Robust central tendency
std_fitness	Population spread
alive_count	Agents alive at generation end
food_consumed	Total resource consumption
avg_lifespan	Mean lifespan
species_count	Number of identified species, if enabled

17.2 Configuration Example

```
{  
  "population_size": 250,  
  ...  
}
```

```
"generation_steps": 5000,
"mutation_rate": 0.05,
"mutation_strength": 0.10,
"elite_fraction": 0.10,
"sensor_count": 7,
"food_count": 80,
"obstacle_count": 12,
"seed": 42
}
```

18. Experimental Methodology

The project should use controlled experiments rather than a single training run. Each experiment changes one primary variable while keeping the remaining configuration fixed. Multiple random seeds are required for conclusions that are robust to initialization noise.

18.1 Experiment A - Mutation Rate

Condition	Mutation rate
A1	0.01
A2	0.05
A3	0.10
A4	0.20

Measure final fitness, convergence generation, variance between seeds, and population diversity.

18.2 Experiment B - Population Size

Condition	Population
B1	50
B2	100
B3	250
B4	500

Measure final fitness and computational cost. The purpose is to quantify the trade-off between search diversity and runtime.

18.3 Experiment C - Network Capacity

Condition	Architecture
C1	12 -> 8 -> 3
C2	12 -> 16 -> 8 -> 3
C3	12 -> 32 -> 16 -> 3

19. Evaluation Plan

19.1 Primary Metrics

Metric	Purpose
Best fitness	Measures strongest discovered strategy

Mean fitness	Measures population-wide improvement
Median fitness	Reduces sensitivity to outliers
Convergence generation	Measures speed of evolution
Survival time	Measures behavioral effectiveness
Food efficiency	Measures resource use per unit energy
Generalization score	Measures performance on unseen worlds

19.2 Baselines

The evaluation should include at least one non-evolutionary baseline. A deterministic hand-coded controller can serve as a reference point for basic task performance, while a randomly initialized untrained neural population provides a no-learning baseline.

19.3 Generalization Test

```

TRAINING
  evolve on layouts A1, A2, A3
    |
    v
FREEZE BEST CONTROLLERS
  |
  v
TEST
  evaluate on unseen layouts B1, B2, B3
    |
    v
  compare train vs unseen performance

```

INTERPRETATION

A strong result is not merely high training fitness. A useful evolutionary controller should retain meaningful performance when the environment layout, resource positions, or obstacle configuration changes.

20. Development Plan

Phase	Main work	Exit criterion
Phase 1	Environment + renderer + basic physics	Agents can move and collide reliably.
Phase 2	Sensors + fixed neural controller	Agents react to observations through NN outputs.
Phase 3	Fitness + genetic algorithm	Fitness improves over controlled runs.
Phase 4	Analytics + checkpoints	Experiments are logged and repeatable.
Phase 5	Generalization experiments	Unseen layouts can be evaluated automatically.
Phase 6	Predator/prey + reproduction	Multi-agent ecosystem is stable.
Phase 7	Species / NEAT extension	Topology evolution works on an isolated benchmark.
Phase 8	Packaging + final report	Demo, documentation, tests, and final experiment results complete.

21. Testing Strategy

21.1 Unit Tests

- Sensor ray intersection and normalization.
- Genome serialization/deserialization.
- Neural-network output dimensions and value ranges.
- Mutation bounds and mutation probability.
- Crossover output length and valid parameter ranges.
- Fitness calculation for known synthetic cases.
- Collision handling and energy updates.

21.2 Integration Tests

- Run one complete generation with a tiny population.
- Checkpoint and restore a genome without changing outputs.
- Reset the environment and verify entity counts.
- Run a seeded experiment twice and compare metrics.

21.3 Performance Tests

- Measure steps per second at population sizes 50, 100, 250, and 500.
- Measure rendering overhead separately from simulation overhead.
- Profile neural inference and collision detection.

22. Risk Analysis

Risk	Impact	Likelihood	Mitigation
Fitness hacking	High	Medium	Use bounded reward terms and behavioral diagnostics.
Premature convergence	High	Medium	Mutation, elite limits, diversity tracking, optional species.
Simulation too slow	High	Medium	Vectorize inference; reduce draw cost; use fast-forward.
Non-reproducible results	High	Medium	Seeds, configs, versioning, deterministic updates.
Feature creep	Medium	High	Freeze MVP before adding predators/species/NEAT.
Visualization distracts from research	Medium	Medium	Keep analytics separate from core simulation logic.

23. Security and Stability

- Validate configuration values before starting an experiment.
- Reject NaN/Inf values in genomes, fitness values, and physics state.
- Cap population size, generation duration, and checkpoint frequency.
- Use safe serialization formats such as JSON for metadata and NumPy-safe arrays for numeric data.
- Keep the simulation sandboxed; evolving agents do not execute arbitrary code.

- Record software version and configuration hash for every experiment.

24. Deployment and Packaging

24.1 Desktop Application

The primary deliverable can be packaged as a Python application with a command-line launcher for experiments and a PyGame interface for live demonstrations.

```
python -m app.run --config configs/baseline.json
python -m experiments.run --config configs/mutation_rate.json --seeds 10
```

24.2 Optional Web Dashboard

A FastAPI backend can expose experiment status and stored metrics, while a React frontend can visualize generations, fitness curves, species counts, and best-agent statistics. This is an optional presentation layer and should not be coupled to the simulation core.

25. Future Extensions

- NEAT-style topology evolution.
- Vision based on rendered pixels instead of handcrafted sensor rays.
- Predator/prey multi-species ecology.
- Dynamic weather, day/night, and changing resources.
- Multi-agent cooperation and competition.
- Evolution of body parameters such as speed and energy efficiency.
- Parallelized population evaluation on CPU/GPU.
- 3D simulation using Godot or a dedicated simulator.
- Distributed experiment execution.
- Behavior cloning or reinforcement-learning comparison baselines.

26. Expected Deliverables

Deliverable	Description
Source code	Modular Python project with tests.
Simulation executable	Interactive artificial-life environment.
Configuration files	Reproducible experiment definitions.
Experiment logs	Generation and agent metrics.
Model checkpoints	Best genomes and network parameters.
Analytics	Fitness and population visualizations.
Technical report	Architecture, algorithms, methodology, and results.
Demonstration	Live simulation showing evolutionary progress.
Optional web dashboard	Browser-based monitoring and experiment visualization.

27. Conclusion

The proposed artificial-life system provides a complete engineering and research framework for studying autonomous behavior through neuroevolution. Its central design principle is separation of

concerns: the environment generates the problem, the neural network generates behavior, the fitness function evaluates consequences, and the genetic algorithm improves the population across generations.

The recommended implementation strategy is to validate a small fixed-topology system first, then add complexity in controlled stages. This approach keeps the project implementable while creating several measurable research dimensions: mutation rate, population size, network capacity, environmental complexity, species diversity, and generalization to unseen environments.

FINAL DESIGN BASELINE

MVP: 2D world + food + obstacles + 100-250 agents + 7-ray sensors + small MLP + fitness + selection + crossover + mutation + analytics. Advanced version: reproduction + predators + species + topology evolution + unseen-environment evaluation.

Appendix A - Recommended Initial Parameters

Parameter	Recommended baseline
World	1200 x 700
Population	250
Generation length	5000 simulation steps
Food	80
Obstacles	12
Sensor rays	7
Max sensor range	200 px
Network	12-32-16-3
Selection	Tournament + 10% elitism
Crossover	Uniform
Mutation rate	0.05
Mutation strength	0.10
Seeds per experiment	10
Fast-forward	Up to 20x

Appendix B - Reference Pseudocode

```
def run_generation(population, world):
    for step in range(world.generation_steps):
        for agent in population:
            if not agent.alive:
                continue

            observation = agent.sensors.observe(world)
            action = agent.network(observation)
            agent.apply_action(action)

        world.update()

    for agent in population:
        if agent.alive:
            agent.update_fitness(world)
```

```
finalize_population(population)

elites = select_elites(population)
parents = select_parents(population)

children = []
while len(children) < len(population) - len(elites):
    a, b = choose_two(parents)
    genome = crossover(a.genome, b.genome)
    genome = mutate(genome)
    children.append(make_agent(genome))

return elites + children
```

Appendix C - Proposed Project Title Variants

- Design and Implementation of an Artificial Life Simulation Using Neuroevolution and Genetic Algorithms
- Evolution of Autonomous Agents Through Neural Networks and Genetic Algorithms in a Dynamic Environment
- Neuroevolution-Based Artificial Life: Design, Simulation, and Experimental Evaluation of Adaptive Agents